

Design and Analysis of Algorithm Knapsack Problem

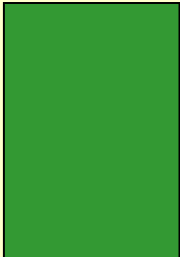
The Knapsack Problem

- ♦ The famous *knapsack problem*:
 - » A thief breaks into a museum. Fabulous paintings, sculptures, and jewels are everywhere. The thief has a good eye for the value of these objects, and knows that each will fetch hundreds or thousands of dollars on the clandestine art collector's market. But, the thief has only brought a single knapsack to the scene of the robbery, and can take away only what he can carry. What items should the thief take to maximize the haul?

0-1 Knapsack problem

- ◆ Given a knapsack with maximum capacity W , and a set S consisting of n items
- ◆ Each item i has some weight w_i and benefit value b_i (all w_i , b_i and W are integer values)
- ◆ Problem: How to pack the knapsack to achieve maximum total value of packed items?

0-1 Knapsack problem: A picture

	<u>Items</u>	<u>Weight</u>	<u>Benefit value</u>
		$\underline{w_i}$	$\underline{b_i}$
<u>This is a knapsack</u> <u>Max weight: $W = 20$</u> <u>$W = 20$</u>		<u>2</u>	<u>3</u>
		<u>3</u>	<u>4</u>
		<u>4</u>	<u>5</u>
		<u>5</u>	<u>8</u>
		<u>9</u>	<u>10</u>

The Knapsack Problem

- ◆ More formally, the *0-1 knapsack problem*:
 - » The thief must choose among n items, where the i th item worth v_i dollars and weighs w_i pounds
 - » Carrying at most W pounds, maximize value
 - Note: assume v_i , w_i , and W are all integers
 - “0-1” because each item must be taken or left in entirety
- ◆ A variation, the *fractional knapsack problem*:
 - » Thief can take fractions of items
 - » Think of items in 0-1 problem as gold ingots, in fractional problem as buckets of gold dust

0-1 Knapsack problem

- ♦ The problem, in other words, is to find

$$\max \sum_{i \in T} b_i \text{ subject to } \sum_{i \in T} w_i \leq W$$

- The problem is called a “0-1” problem, because each item must be entirely accepted or rejected.
- Just another version of this problem is the “Fractional Knapsack Problem”, where we can take fractions of items.

0-1 Knapsack problem: brute-force approach

Let's first solve this problem with a straightforward algorithm

- ◆ Since there are n items, there are 2^n possible combinations of items.
- ◆ We go through all combinations and find the one with the most total value and with total weight less or equal to W
- ◆ Running time will be $O(2^n)$

0-1 Knapsack problem: brute-force approach

- ♦ Can we do better?
- ♦ Yes, with an algorithm based on dynamic programming
- ♦ We need to carefully identify the subproblems

Let's try this:

If items are labeled $1..n$, then a subproblem would be to find an optimal solution for $S_k = \{items\ labeled\ 1, 2, .. k\}$

Recursive Formula for subproblems

■ Recursive formula for subproblems:

$$B[k, w] = \begin{cases} B[k - 1, w] & \text{if } w_k > w \\ \max \{B[k - 1, w], B[k - 1, w - w_k] + b_k\} & \text{else} \end{cases}$$

- ◆ It means, that the best subset of S_k that has total weight w is one of the two:
 - 1) the best subset of S_{k-1} that has total weight w , **or**
 - 2) the best subset of S_{k-1} that has total weight $w - w_k$ plus the item k

Recursive Formula

$$B[k, w] = \begin{cases} B[k - 1, w] & \text{if } w_k > w \\ \max \{B[k - 1, w], B[k - 1, w - w_k] + b_k\} & \text{else} \end{cases}$$

- ◆ The best subset of S_k that has the total weight w , either contains item k or not.
- ◆ First case: $w_k > w$. Item k can't be part of the solution, since if it was, the total weight would be $> w$, which is unacceptable
- ◆ Second case: $w_k \leq w$. Then the item k can be in the solution, and we choose the case with greater value

0-1 Knapsack Algorithm

for $w = 0$ to W

$$B[0,w] = 0$$

for $i = 0$ to n

$$B[i,0] = 0$$

for $w = 0$ to W

if $w_i \leq w$ // item i can be part of the solution

$$\text{if } b_i + B[i-1, w-w_i] > B[i-1, w]$$

$$B[i, w] = b_i + B[i-1, w-w_i]$$

else

$$B[i, w] = B[i-1, w]$$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Running time

- The running time of the Knapsack procedure is $O(nW)$, since each table entry takes $O(1)$ time to compute.
- It is substantially better than the exponential running time of the brute force method.

Remember that the brute-force algorithm
takes $O(2^n)$

Example

Let's run our algorithm on the following data:

$n = 4$ (No. of elements)

$W = 5$ (max weight)

Elements (weight, benefit):

$(2,3), (3,4), (4,5), (5,6)$

Example (2)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>	<u>0</u>					
<u>1</u>	<u>0</u>					
<u>2</u>	<u>0</u>					
<u>3</u>	<u>0</u>					
<u>4</u>	<u>0</u>					
<u>5</u>	<u>0</u>					

for w = 0 to W

B[0,w] = 0

Example (3)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>	<u>0</u>					
<u>2</u>	<u>0</u>					
<u>3</u>	<u>0</u>					
<u>4</u>	<u>0</u>					
<u>5</u>	<u>0</u>					

for i = 0 to n
B[i,0] = 0

Example (4)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u> → <u>0</u>			
<u>2</u>		<u>0</u>				
<u>3</u>		<u>0</u>				
<u>4</u>		<u>0</u>				
<u>5</u>		<u>0</u>				

$$\underline{i=1}$$

$$\underline{b_i=3}$$

$$\underline{w_i=2}$$

$$\underline{w=1}$$

$$\underline{w-w_i=-1}$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + B[i-1, w-w_i] > B[i-1, w]$

$B[i, w] = b_i + B[i-1, w-w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Example (5)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>			
<u>2</u>		<u>0</u>	<u>3</u>			
<u>3</u>		<u>0</u>				
<u>4</u>		<u>0</u>				
<u>5</u>		<u>0</u>				

$$\underline{i=1}$$

$$\underline{b_i=3}$$

$$\underline{w_i=2}$$

$$\underline{w=2}$$

$$\underline{w-w_i=0}$$

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $\underline{w_i} \leq w$ // item i can be part of the solution

if $\underline{b_i} + B[i-1, w-w_i] > B[i-1, w]$

$B[i, w] = b_i + B[i-1, w-w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $\underline{w_i} > w$

Example (6)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>			
<u>2</u>		<u>0</u>	<u>3</u>			
<u>3</u>		<u>0</u>	<u>3</u>			
<u>4</u>		<u>0</u>				
<u>5</u>		<u>0</u>				

i=1

b_i=3

w_i=2

w=3

w-w_i=1

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

if b_i + B[i-1,w-w_i] > B[i-1,w]

B[i,w] = b_i + B[i-1,w-w_i]

else

B[i,w] = B[i-1,w]

else B[i,w] = B[i-1,w] // w_i > w

Example (7)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>			
<u>2</u>		<u>0</u>	<u>3</u>			
<u>3</u>		<u>0</u>	<u>3</u>			
<u>4</u>		<u>0</u>	<u>3</u>			
<u>5</u>		<u>0</u>				

i=1

b_i=3

w_i=2

w=4

w-w_i=2

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i ≤ w // item i can be part of the solution

if b_i + B[i-1, w-w_i] > B[i-1, w]

B[i, w] = b_i + B[i-1, w-w_i]

else

B[i, w] = B[i-1, w]

else B[i, w] = B[i-1, w] // w_i > w

Example (8)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>			
<u>2</u>		<u>0</u>	<u>3</u>			
<u>3</u>		<u>0</u>	<u>3</u>			
<u>4</u>		<u>0</u>	<u>3</u>			
<u>5</u>		<u>0</u>	<u>3</u>			

i=1

b_i=3

w_i=2

w=5

w-w_i=2

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i ≤ w // item i can be part of the solution

if b_i + B[i-1, w-w_i] > B[i-1, w]

B[i, w] = b_i + B[i-1, w-w_i]

else

B[i, w] = B[i-1, w]

else B[i, w] = B[i-1, w] // w_i > w

Example (9)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>	<u>0</u> →		
<u>2</u>		<u>0</u>	<u>3</u>			
<u>3</u>		<u>0</u>	<u>3</u>			
<u>4</u>		<u>0</u>	<u>3</u>			
<u>5</u>		<u>0</u>	<u>3</u>			

i=2

b_i=4

w_i=3

w=1

w-w_i=-2

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + B[i-1, w-w_i] > B[i-1, w]$

$B[i, w] = b_i + B[i-1, w-w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Example (10)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>	<u>0</u>		
<u>2</u>		<u>0</u>	<u>3</u>	<u>3</u>		
<u>3</u>		<u>0</u>	<u>3</u>			
<u>4</u>		<u>0</u>	<u>3</u>			
<u>5</u>		<u>0</u>	<u>3</u>			

i=2

b_i=4

w_i=3

w=2

w-w_i=-1

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + B[i-1, w-w_i] > B[i-1, w]$

$B[i, w] = b_i + B[i-1, w-w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Example (11)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>		
<u>2</u>	<u>0</u>	<u>3</u>	<u>3</u>			
<u>3</u>	<u>0</u>	<u>3</u>	<u>4</u>			
<u>4</u>	<u>0</u>	<u>3</u>				
<u>5</u>	<u>0</u>	<u>3</u>				

i=2

b_i=4

w_i=3

w=3

w-w_i=0

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i ≤ w // item i can be part of the solution

if b_i + B[i-1, w-w_i] > B[i-1, w]

B[i, w] = b_i + B[i-1, w-w_i]

else

B[i, w] = B[i-1, w]

else B[i, w] = B[i-1, w] // w_i > w

Example (12)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>			
<u>2</u>	<u>0</u>	<u>3</u>	<u>3</u>			
<u>3</u>	<u>0</u>	<u>3</u>	<u>4</u>			
<u>4</u>	<u>0</u>	<u>3</u>	<u>4</u>			
<u>5</u>	<u>0</u>	<u>3</u>				

i=2

b_i=4

w_i=3

w=4

w-w_i=1

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

if b_i + B[i-1, w-w_i] > B[i-1, w]

B[i, w] = b_i + B[i-1, w-w_i]

else

B[i, w] = B[i-1, w]

else B[i, w] = B[i-1, w] // w_i > w

Example (13)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>	<u>0</u>		
<u>2</u>		<u>0</u>	<u>3</u>	<u>3</u>		
<u>3</u>		<u>0</u>	<u>3</u>	<u>4</u>		
<u>4</u>		<u>0</u>	<u>3</u>	<u>4</u>		
<u>5</u>		<u>0</u>	<u>3</u>	<u>7</u>		

i=2

b_i=4

w_i=3

w=5

w-w_i=2

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i <= w // item i can be part of the solution

if b_i + B[i-1,w-w_i] > B[i-1,w]

B[i,w] = b_i + B[i-1,w-w_i]

else

B[i,w] = B[i-1,w]

else B[i,w] = B[i-1,w] // w_i > w

Example (14)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>	<u>0</u> → <u>0</u>		
<u>2</u>		<u>0</u>	<u>3</u>	<u>3</u> → <u>3</u>		
<u>3</u>		<u>0</u>	<u>3</u>	<u>4</u> → <u>4</u>		
<u>4</u>		<u>0</u>	<u>3</u>	<u>4</u>		
<u>5</u>		<u>0</u>	<u>3</u>	<u>7</u>		

i=3

b_i=5

w_i=4

w=1..3

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + B[i-1, w-w_i] > B[i-1, w]$

$B[i, w] = b_i + B[i-1, w-w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Example (15)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	
<u>2</u>	<u>0</u>	<u>3</u>	<u>3</u>	<u>3</u>		
<u>3</u>	<u>0</u>	<u>3</u>	<u>4</u>	<u>4</u>		
<u>4</u>	<u>0</u>	<u>3</u>	<u>4</u>	<u>5</u>		
<u>5</u>	<u>0</u>	<u>3</u>	<u>7</u>			

i=3

b_i=5

w_i=4

w=4

w - w_i=0

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i ≤ w // item i can be part of the solution

if b_i + B[i-1, w-w_i] > B[i-1, w]

B[i, w] = b_i + B[i-1, w - w_i]

else

B[i, w] = B[i-1, w]

else B[i, w] = B[i-1, w] // w_i > w

Example (15)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	
<u>2</u>		<u>0</u>	<u>3</u>	<u>3</u>	<u>3</u>	
<u>3</u>		<u>0</u>	<u>3</u>	<u>4</u>	<u>4</u>	
<u>4</u>		<u>0</u>	<u>3</u>	<u>4</u>	<u>5</u>	
<u>5</u>		<u>0</u>	<u>3</u>	<u>7</u> → <u>7</u>		

i=3

b_i=5

w_i=4

w=5

w - w_i=1

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i ≤ w // item i can be part of the solution

if b_i + B[i-1, w-w_i] > B[i-1, w]

B[i, w] = b_i + B[i-1, w - w_i]

else

B[i, w] = B[i-1, w]

else B[i, w] = B[i-1, w] // w_i > w

Example (16)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u> →	<u>0</u>
<u>2</u>		<u>0</u>	<u>3</u>	<u>3</u>	<u>3</u> →	<u>3</u>
<u>3</u>		<u>0</u>	<u>3</u>	<u>4</u>	<u>4</u> →	<u>4</u>
<u>4</u>		<u>0</u>	<u>3</u>	<u>4</u>	<u>5</u> →	<u>5</u>
<u>5</u>		<u>0</u>	<u>3</u>	<u>7</u>	<u>7</u>	

i=4

b_i=6

w_i=5

w=1..4

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if $w_i \leq w$ // item i can be part of the solution

if $b_i + B[i-1, w-w_i] > B[i-1, w]$

$B[i, w] = b_i + B[i-1, w-w_i]$

else

$B[i, w] = B[i-1, w]$

else $B[i, w] = B[i-1, w]$ // $w_i > w$

Example (17)

<u>W</u>	<u>i</u>	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
<u>0</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>1</u>		<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>
<u>2</u>		<u>0</u>	<u>3</u>	<u>3</u>	<u>3</u>	<u>3</u>
<u>3</u>		<u>0</u>	<u>3</u>	<u>4</u>	<u>4</u>	<u>4</u>
<u>4</u>		<u>0</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>5</u>
<u>5</u>		<u>0</u>	<u>3</u>	<u>7</u>	<u>7</u>	<u>7</u>

i=4

b_i=6

w_i=5

w=5

w - w_i=0

Items:

1: (2,3)

2: (3,4)

3: (4,5)

4: (5,6)

if w_i ≤ w // item i can be part of the solution

if b_i + B[i-1,w-w_i] > B[i-1,w]

B[i,w] = b_i + B[i-1,w-w_i]

else

B[i,w] = B[i-1,w]

else B[i,w] = B[i-1,w] // w_i > w

Comments

- ♦ This algorithm only finds the max possible value that can be carried in the knapsack
- ♦ To know the items that make this maximum value, an addition to this algorithm is necessary
- ♦ Please see LCS algorithm from the previous lecture for the example how to extract this data from the table we built

Dynamic Programming

- ♦ Dynamic programming is a useful technique of solving certain kind of problems
- ♦ When the solution can be recursively described in terms of partial solutions, we can store these partial solutions and re-use them as necessary
- ♦ Running time (Dynamic Programming algorithm vs. naïve algorithm):
 - » LCS: **$O(m * n)$** vs. **$O(n * 2^m)$**
 - » 0-1 Knapsack problem: **$O(W * n)$** vs. **$O(2^n)$**